Исследование клиентов банка

Исследование выполнила Коробкова Мария
в рамках курса по анализу данных

Распределение и размах величин CustAccountBalance и TransactionAmount

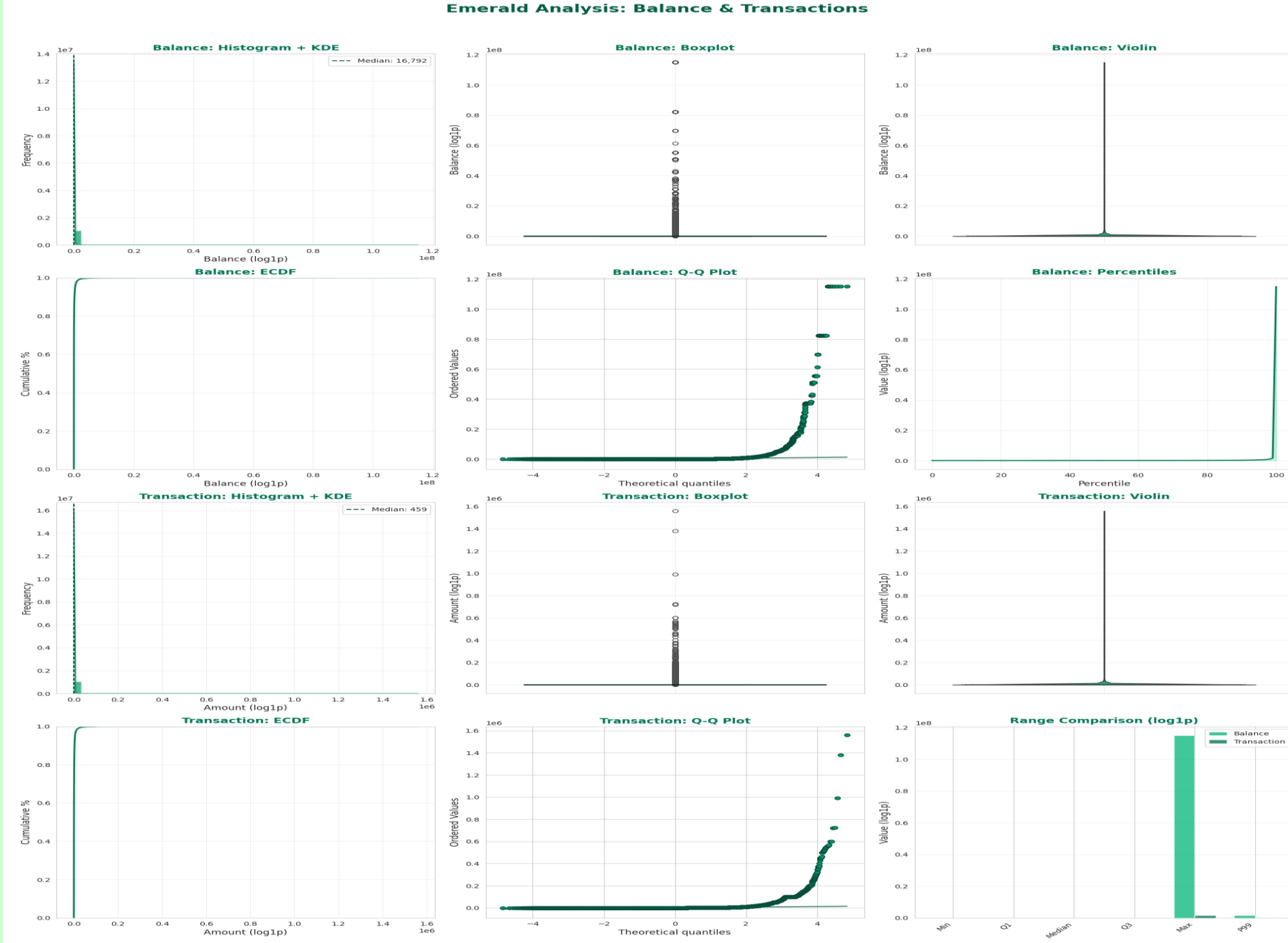
```python
# изучить распределение и размах величин CustAccountBalance и TransactionAmount

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import skew, kurtosis, probplot  # <--- ДОБАВИТЬ ЭТУ СТРОКУ

# Настройка стиля
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (16, 10)
plt.rcParams['font.size'] = 10

# ## Изумрудная палитра (Emerald Palette)

EMERALD = {
    '50': '#ECFDF5', '100': '#D1FAE5', '200': '#A7F3D0', '300': '#6EE7B7',
    '400': '#34D399', '500': '#10B981', '600': '#059669', '700': '#047857',
    '800': '#065F46', '900': '#064E3B', '950': '#022C22'
}

# ## Функция анализа

def analyze_column(df, col_name, title, log_transform=False):
    """Базовый анализ"""
    data = df[col_name].dropna()

    # Применяем логарифмирование для графиков
    if log_transform and (data > 0).all():
        data_plot = np.log1p(data)
        suffix = " (log1p)"
        # Статистики считаем по исходным данным для бизнес-отчётности
        data_for_stats = data
    else:
        data_plot = data
        data_for_stats = data
        suffix = ""
```

```python
# Считаем промежуточные переменные
q1 = data_for_stats.quantile(0.25)
q3 = data_for_stats.quantile(0.75)
iqr = q3 - q1

lower_bound = q1 - 1.5 * iqr
upper_bound = q3 + 1.5 * iqr

outliers_count = len(data_for_stats[(data_for_stats < lower_bound) |
                                    (data_for_stats > upper_bound)])


# Теперь собираем словарь
stats_dict = {
    'Count': len(data_for_stats),
    'Mean': data_for_stats.mean(),
    'Median': data_for_stats.median(),
    'Std': data_for_stats.std(),
    'Min': data_for_stats.min(),
    'Max': data_for_stats.max(),
    'Range': data_for_stats.max() - data_for_stats.min(),
    'Q1': q1,
    'Q3': q3,
    'IQR': iqr,
    'Skewness': skew(data_for_stats),
    'Kurtosis': kurtosis(data_for_stats),
    'P95': data_for_stats.quantile(0.95),
    'P99': data_for_stats.quantile(0.99),
    'Outliers_IQR': outliers_count
}

print(f" {title}{suffix}")

for k, v in stats_dict.items():
    if isinstance(v, float):
        print(f"{k:15}: {v:,.2f}")
    else:
        print(f"{k:15}: {v:,}")

return data_plot, stats_dict
```

```python
# Копируем df для безопасности
df_work = df.copy()

# Фильтрация некорректных значений (убираем отрицательные балансы и суммы)
df_work = df_work[(df_work['custAccountBalance'] >= 0) &
                  (df_work['transactionAmountInr'] >= 0)]

print(f" Обработано записей: {len(df_work):,}")


# ## Анализ custAccountBalance

balance_data, balance_stats = analyze_column(
    df_work, 'custAccountBalance', 'Баланс счёта клиента', log_transform=True
)

# ## Анализ transactionAmountInr

txn_data, txn_stats = analyze_column(
    df_work, 'transactionAmountInr', 'Сумма транзакции', log_transform=True
)

# ## Визуализация: Изумрудная галерея

fig = plt.figure(figsize=(20, 24))
fig.suptitle('Emerald Analysis: Balance & Transactions',
             fontsize=20, fontweight='bold', color=EMERALD['800'], y=1.02)
```

Распределение и размах величин CustAccountBalance и TransactionAmount

Распределения баланса и сумм транзакций демонстрируют экстремальную правостороннюю асимметрию (skewness 60.59 и 47.34 соответственно от нормальной симметрии) с чрезвычайно тяжёлыми хвостами (kurtosis > 5000), что указывает на наличие небольшой доли клиентов с очень высокими показателями на фоне массовой концентрации вокруг низких значений.



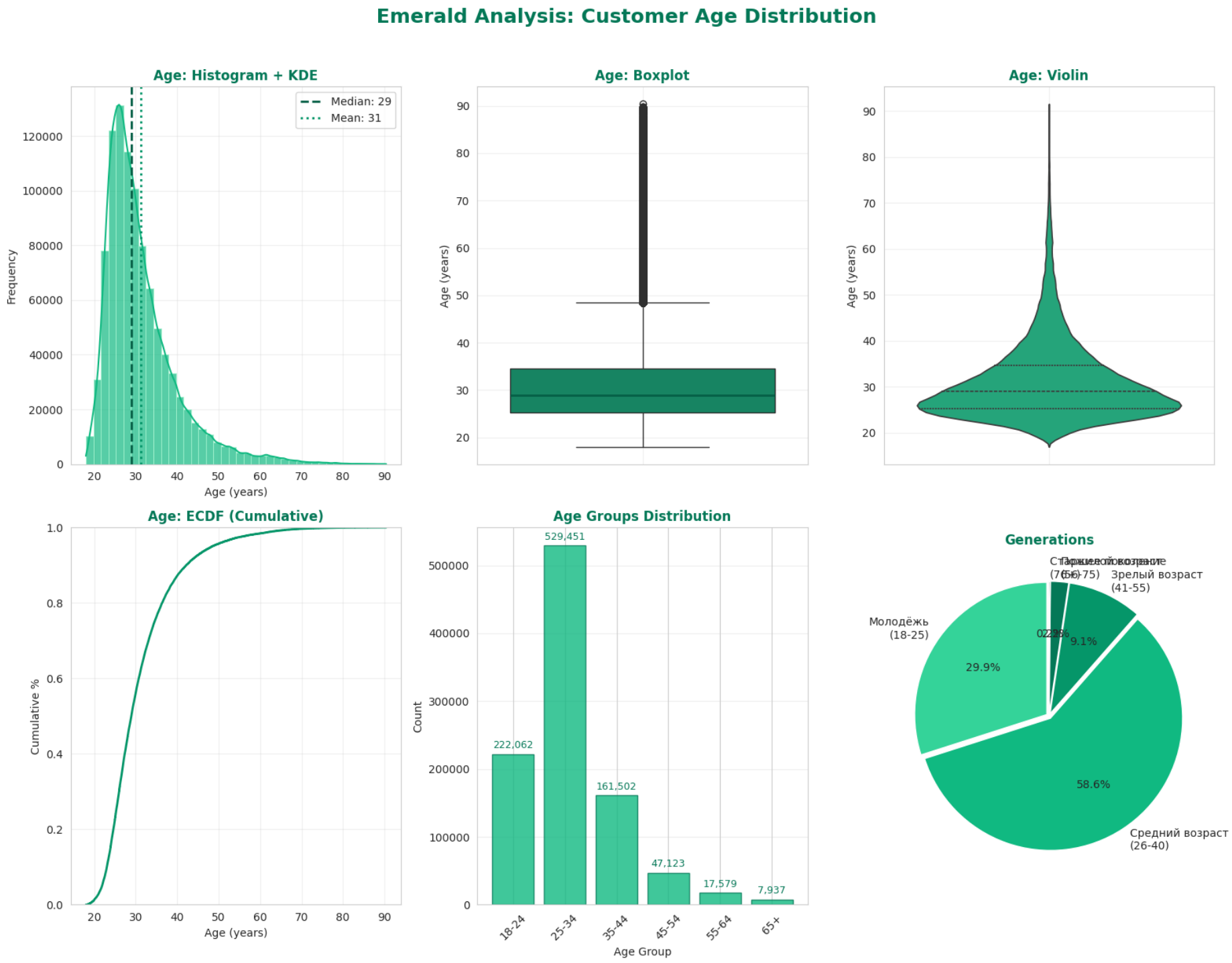
Анализ и визуализация возраста клиентов

```python
# 1. ПОДГОТОВКА ДАННЫХ

print(" Расчет возраста клиентов...")

df_work = df.copy()

# Преобразуем customerDob в datetime
df_work['customerDob'] = pd.to_datetime(df_work['customerDob'], errors='coerce')

#  Возраст на дату транзакции
df_work['age_at_transaction'] = (df_work['transactionDate'] - df_work['customerDob']).dt.days / 365.25

# Текущий возраст
reference_date = pd.Timestamp('2026-02-17')  # Сегодня
df_work['age'] = (reference_date - df_work['customerDob']).dt.days / 365.25

# Фильтруем корректный возраст (18-100 лет)
df_age = df_work[(df_work['age_at_transaction'] >= 18) &
                 (df_work['age_at_transaction'] <= 100)].copy()

# Статистика пропусков
missing_dob = df['isDobMissing'].sum()
total = len(df)

print(f" Всего записей: {total:,}")
print(f" Пропуски DOB: {missing_dob:,} ({missing_dob/total*100:.1f}%)")
print(f" Записей с возрастом: {len(df_age):,} ({len(df_age)/total*100:.1f}%)")
```

```python
# 2. АНАЛИЗ РАСПРЕДЕЛЕНИЯ ВОЗРАСТА

ages = df_age['age_at_transaction']

q1, q3 = ages.quantile([0.25, 0.75])
iqr = q3 - q1
outliers = len(ages[(ages < q1 - 1.5*iqr) | (ages > q3 + 1.5*iqr)])

print(" СТАТИСТИКА ВОЗРАСТА")

print(f"Mean:          {ages.mean():>10.1f} лет")
print(f"Median:        {ages.median():>10.1f} лет")
print(f"Std:           {ages.std():>10.1f}")
print(f"Min:           {ages.min():>10.1f} лет")
print(f"Max:           {ages.max():>10.1f} лет")
print(f"Q1 (25%):      {q1:>10.1f} лет")
print(f"Q3 (75%):      {q3:>10.1f} лет")
print(f"IQR:           {iqr:>10.1f}")
print(f"Skewness:      {skew(ages):>10.2f}")
print(f"Kurtosis:      {kurtosis(ages):>10.2f}")
print(f"Outliers:      {outliers:>10,} ({outliers/len(ages)*100:.1f}%)")
```

Анализ и визуализация возраст клиентов

База клиентов сконцентрирована вокруг молодой аудитории: медианный возраст составляет 29 лет, а 88,5% клиентов — это люди до 40 лет, из которых половина (50%) находится в возрасте 25-34 лет.



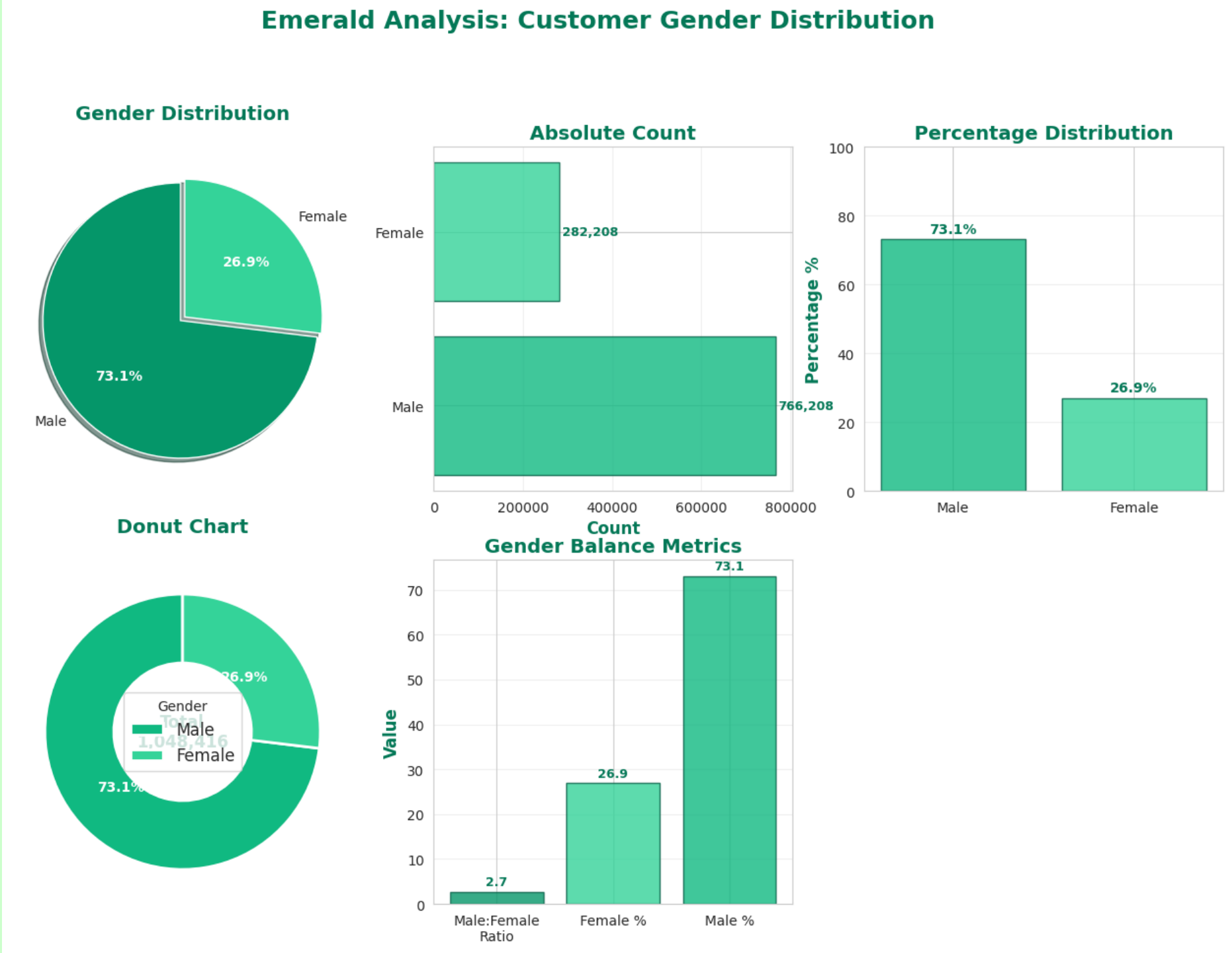
Анализ и визуализация гендера клиентов

```python
# 1. ПОДГОТОВКА ДАННЫХ

print(" Анализ распределения по полу...")

df_work = df.copy()
# Словарь для нормализации значений пола
gender_map = {
    'M': 'Male', 'm': 'Male', 'Male': 'Male', 'MALE': 'Male', 'male': 'Male',
    'F': 'Female', 'f': 'Female', 'Female': 'Female', 'FEMALE': 'Female', 'female': 'Female',
    'U': 'Unknown', 'u': 'Unknown', 'Unknown': 'Unknown', '': 'Unknown', ' ': 'Unknown'
}

df_work['custGender'] = df_work['custGender'].map(lambda x: gender_map.get(str(x).strip(), x))

# Статистика
total = len(df_work)
gender_counts = df_work['custGender'].value_counts()
gender_pct = df_work['custGender'].value_counts(normalize=True) * 100

print(f"Всего клиентов: {total:,}")
print(f"\n Абсолютные значения:")
for gender, count in gender_counts.items():
    print(f"   {gender:12}: {count:>12,} ({count/total*100:>5.1f}%)")
```

```python
# 3. КЛЮЧЕВЫЕ ИНСАЙТЫ

print("Анализ клиентов по возрасту")

# Доминирующий пол
dominant_gender = gender_counts.idxmax()
dominant_pct = gender_pct.max()
print(f" Доминирующий пол: {dominant_gender} ({dominant_pct:.1f}%)")

# Гендерный баланс
if len(gender_counts) >= 2:
    male = gender_counts.get('Male', 0)
    female = gender_counts.get('Female', 0)
    total_mf = male + female

    if total_mf > 0:
        male_pct = male / total_mf * 100
        female_pct = female / total_mf * 100
        ratio = male / female if female > 0 else float('inf')

        print(f" Соотношение Male:Female = {ratio:.2f}:1")
        print(f" Мужчины: {male:,} ({male_pct:.1f}%)")
        print(f" Женщины: {female:,} ({female_pct:.1f}%)")

        if abs(male_pct - 50) < 5:
            print(f" Гендерный баланс близок к равномерному")
        elif male_pct > 60:
            print(f" Явное преобладание мужской аудитории")
        elif female_pct > 60:
            print(f" Явное преобладание женской аудитории")
```

Анализ и визуализация гендера клиентов

Клиентская база демонстрирует гендерный дисбаланс: мужчины составляют большинство клиентов банка — 73.1% (766,208 человек), в то время как женщины представлены значительно меньше — 26.9% (282,208 человек).



Анализ локации клиентов

```python
# 3. Процентное распределение (Топ-10)
ax3 = plt.subplot(3, 3, 3)
top10_pct = location_pct.head(10)
bars_pct = ax3.bar(top10_pct.index, top10_pct.values,
                   color=[EMERALD[3], EMERALD[4], EMERALD[2]]*4,
                   edgecolor=EMERALD[0], alpha=0.8)
ax3.set_ylabel('Percentage %', fontweight='bold', color=EMERALD[1])
ax3.set_title('Top 10: Percentage', fontweight='bold', color=EMERALD[1])
ax3.tick_params(axis='x', rotation=45, labelsize=8)
ax3.grid(alpha=0.3, axis='y')

for bar, pct in zip(bars_pct, top10_pct.values):
    ax3.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.3,
             f'{pct:.1f}%', ha='center', va='bottom', fontsize=8, color=EMERALD[1], fontweight='bold')
```

```python
# 5. Кумулятивное распределение (Pareto)
ax5 = plt.subplot(3, 3, 5)
cumulative_pct = location_counts.head(20).cumsum() / total * 100
ax5.plot(range(1, 21), cumulative_pct.values, marker='o', color=EMERALD[2], linewidth=2)
ax5.axhline(y=80, color=EMERALD[0], linestyle='--', label='80% threshold', alpha=0.7)
ax5.set_xlabel('Location Rank', fontweight='bold', color=EMERALD[1])
ax5.set_ylabel('Cumulative %', fontweight='bold', color=EMERALD[1])
ax5.set_title('Pareto: Cumulative Distribution', fontweight='bold', color=EMERALD[1])
ax5.grid(alpha=0.3)
ax5.legend(fontsize=8)
```

```
 СТАТИСТИКА ПО ЛОКАЦИЯМ
Всего записей: 1,048,416
Уникальных локаций: 9355

Топ-10 локаций по количеству клиентов:
№   Локация                    Клиентов      Доля
--------------------------------------------------
1   MUMBAI                      103,595      9.9%
2   NEW DELHI                    84,928      8.1%
3   BANGALORE                    81,555      7.8%
4   GURGAON                      73,818      7.0%
5   DELHI                        71,019      6.8%
6   NOIDA                        32,784      3.1%
7   CHENNAI                      30,009      2.9%
8   PUNE                         25,851      2.5%
9   HYDERABAD                    23,049      2.2%
10  THANE                        21,505      2.1%

 Концентрация:
   Топ-5 локаций: 39.6% всех клиентов
   Топ-10 локаций: 52.3% всех клиентов
```

Анализ локации клиентов

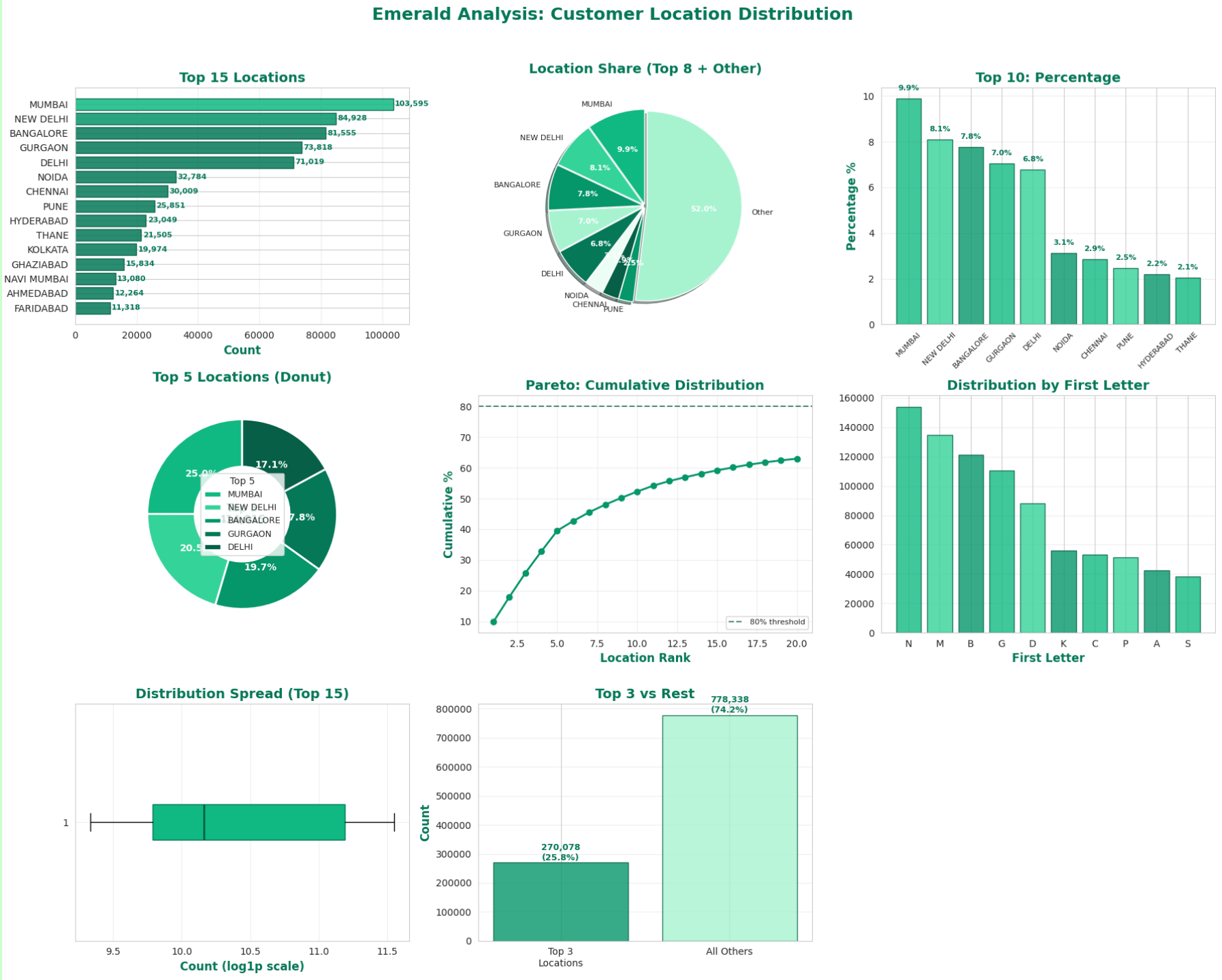
Лидер: MUMBAI – 103,595 клиентов (9.9%)

Умеренная концентрация: топ-5 локаций = 39.6% базы

Выраженный «длинный хвост»: 9261 локаций с долей <0.1%

Анализ локации клиентов

Клиентская база охватывает всю Индию. Крупнейшие мегаполисы - это топ-5-ти локаций (Mumbai, New Delhi, Bangalore, Gurgaon, Delhi) аккумулируют 25.8% всех клиентов, при этом Mumbai лидирует с 103 595 клиентами (9.9%)



Динамика транзакций по дате (в кол-ве TransactionID)

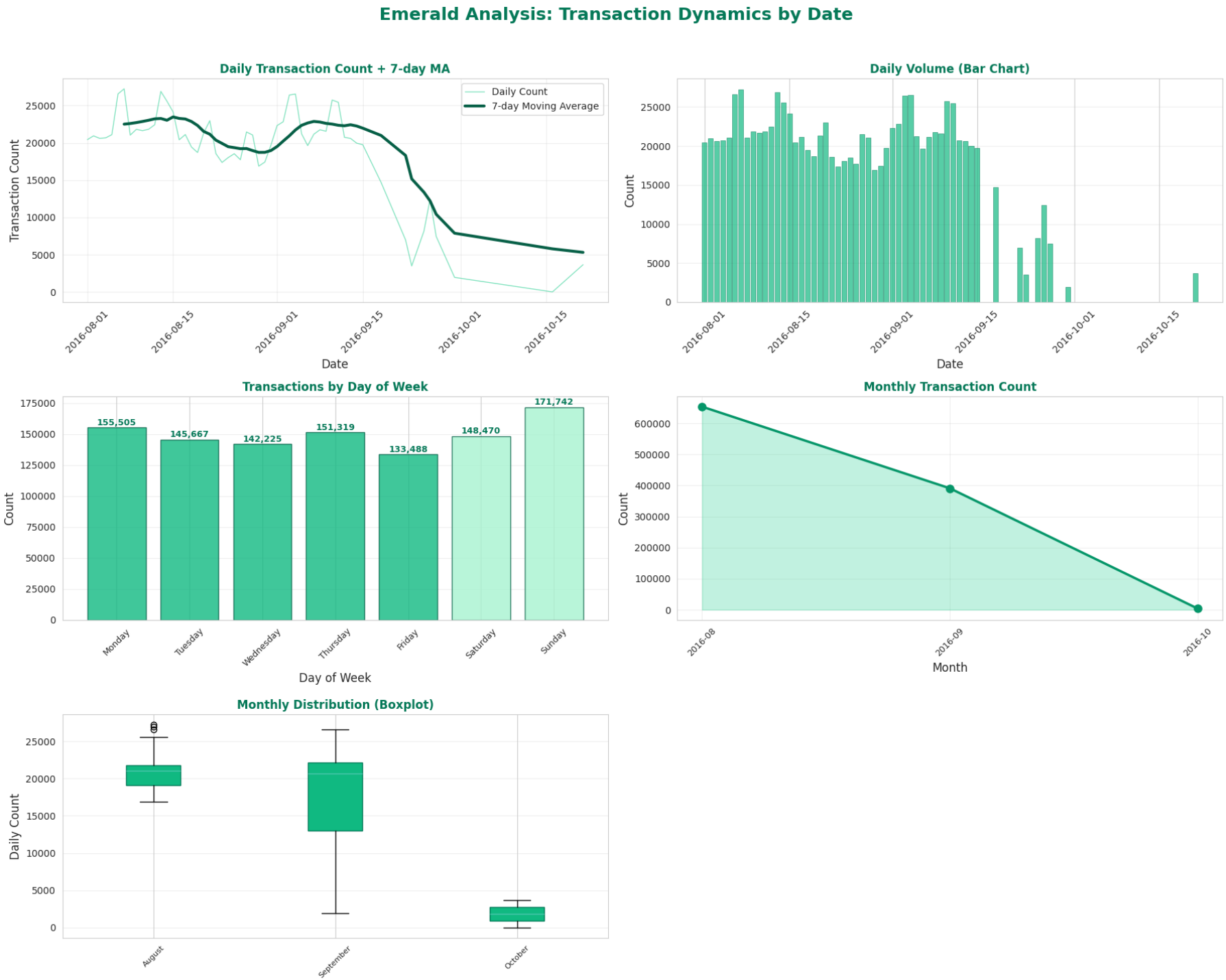
```python
# посмотреть динамику транзакций по дате (в кол-ве TransactionID)

# 1. ПОДГОТОВКА ДАННЫХ

print(" Анализ динамики транзакций...")

# Извлекаем дату (без времени)
df_work['transactionDate'] = df_work['transactionDateTime'].dt.date

# Агрегация по датам
daily_txn = df_work.groupby('transactionDate').agg({
    'transactionId': 'count',
    'transactionAmountInr': ['sum', 'mean']
}).round(2)

daily_txn.columns = ['txn_count', 'total_amount', 'avg_amount']
daily_txn = daily_txn.reset_index()
daily_txn['transactionDate'] = pd.to_datetime(daily_txn['transactionDate'])

# Скользящее среднее (7 дней)
daily_txn['rolling_7d'] = daily_txn['txn_count'].rolling(window=7).mean()

# Статистика
total_days = len(daily_txn)
total_txn = len(df_work)
avg_daily_txn = total_txn / total_days
period_start = daily_txn['transactionDate'].min()
period_end = daily_txn['transactionDate'].max()


print(" ВРЕМЕННАЯ СТАТИСТИКА")

print(f"Период: {period_start.date()} — {period_end.date()}")
print(f"Всего дней: {total_days}")
print(f"Всего транзакций: {total_txn:,}")
print(f"Среднее в день: {avg_daily_txn:,.0f}")
print(f"\n Топ-5 дней по активности:")
top5_days = daily_txn.nlargest(5, 'txn_count')
for i, row in top5_days.iterrows():
    print(f"   {row['transactionDate'].date()}: {row['txn_count']:,.0f} транзакций")
```

```
Анализ динамики сумм транзакций...
 ФИНАНСОВАЯ СТАТИСТИКА
Период: 2016-08-01 — 2016-10-21
Всего дней: 55
Общий объём транзакций: ₹1,650,553,207.16
Средний дневной оборот: ₹30,010,058.31
Средняя транзакция: ₹1,574.33

 Топ-5 дней по объёму:
   2016-08-06: ₹47,518,684.32 (26,578 транзакций)
   2016-09-04: ₹46,116,385.52 (26,560 транзакций)
   2016-08-14: ₹45,730,573.10 (25,592 транзакций)
   2016-08-07: ₹45,718,691.63 (27,254 транзакций)
   2016-09-03: ₹45,292,542.37 (26,421 транзакций)
```

```
 EMERALD INSIGHTS: AMOUNT DYNAMICS
 Пик оборота: 2016-08-07 (₹45.72M)
 Минимум: 2016-10-16 (₹0.00M)
 Лучший день недели: Sunday (₹281.06M)
 Самый тихий день: Friday (₹208.62M)
 Спад оборота: -24.1% (вторая половина периода)
 Высокая волатильность: CV=0.34 (неравномерный денежный поток)

 Средний чек: ₹1,574.33
 Медианный чек: ₹430.00
```

Динамика транзакций по дате (в кол-ве TransactionID)

Наблюдается нисходящий тренд транзакционной активности: за период с августа по октябрь 2016 года среднесуточное количество транзакций сократилось с ~20-27 тысяч до единичных значений (пик 27 254 транзакций 7 августа против 3 транзакций 16 октября).



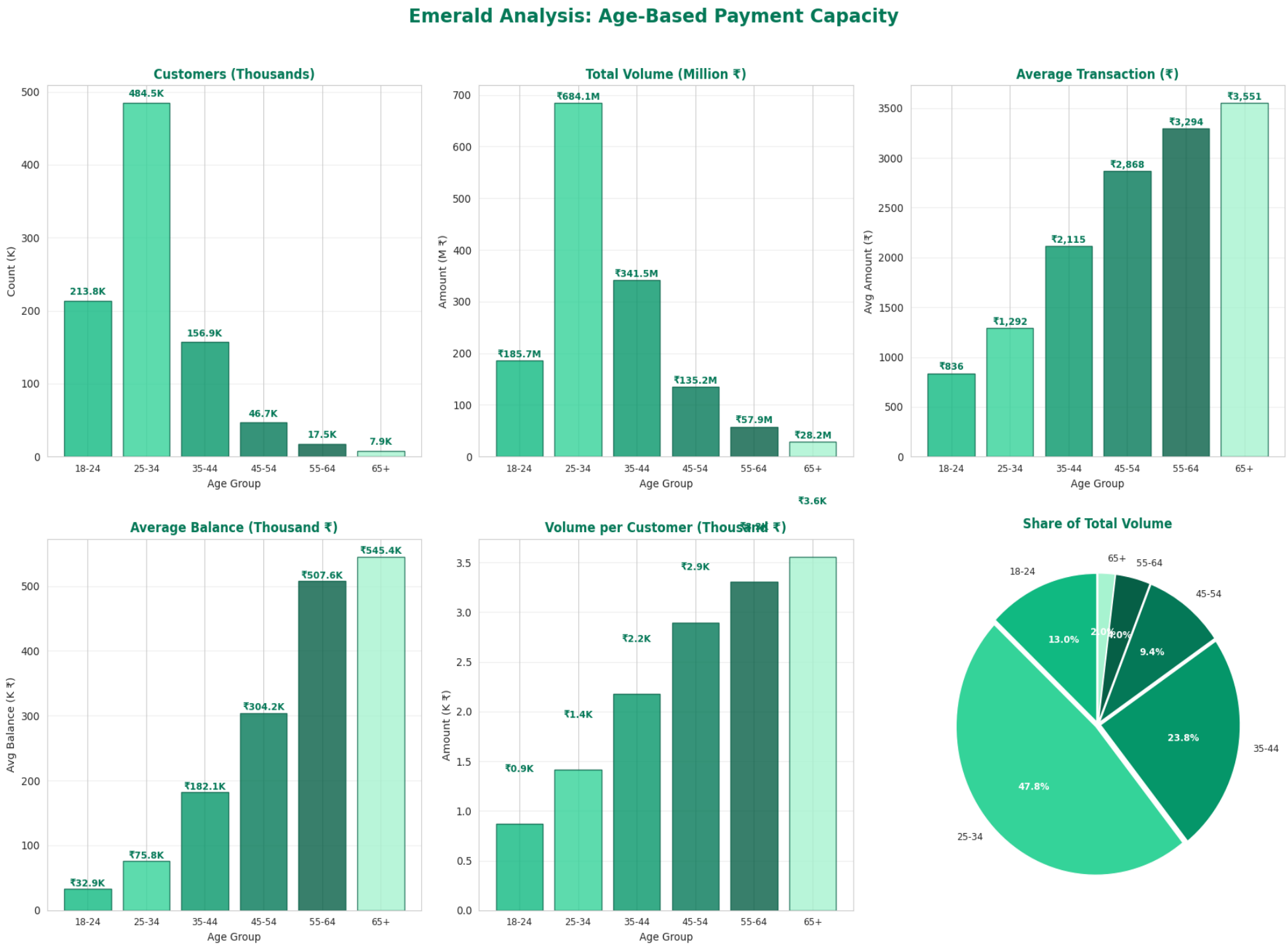
Определение самой платежеспособной группы

```python
# 1. ПОДГОТОВКА ДАННЫХ

print(" Сегментация клиентов по возрасту...")

df_work = df.copy()

# Конвертируем только customerDob
df_work['customerDob'] = pd.to_datetime(df_work['customerDob'], errors='coerce')
df_work['transactionDateTime'] = pd.to_datetime(df_work['transactionDateTime'], errors='coerce')

# Извлекаем дату из transactionDateTime для расчёта возраста
df_work['transactionDate_only'] = df_work['transactionDateTime'].dt.date

# Рассчитываем возраст на дату транзакции
df_work['age_at_transaction'] = (df_work['transactionDateTime'] - df_work['customerDob']).dt.days / 365.25

# 2. ВОЗРАСТНЫЕ КОРЗИНЫ

age_bins = [18, 25, 35, 45, 55, 65, 100]
age_labels = ['18-24', '25-34', '35-44', '45-54', '55-64', '65+']

df_work['age_group'] = pd.cut(df_work['age_at_transaction'],
                              bins=age_bins,
                              labels=age_labels,
                              right=True)
```

```python
# 3. АГРЕГАЦИЯ ПО ГРУППАМ

age_analysis = df_work.groupby('age_group').agg({
    'customerId': 'nunique',
    'transactionId': 'count',
    'transactionAmountInr': ['sum', 'mean', 'median'],
    'custAccountBalance': ['mean', 'median']
}).round(2)

age_analysis.columns = ['unique_customers', 'txn_count', 'total_amount',
                        'avg_txn_amount', 'median_txn_amount',
                        'avg_balance', 'median_balance']

# Дополнительные метрики
age_analysis['total_amount_million'] = age_analysis['total_amount'] / 1e6
age_analysis['amount_per_customer'] = age_analysis['total_amount'] / age_analysis['unique_customers']


print(" АНАЛИЗ ПЛАТЕЖЕСПОСОБНОСТИ ПО ВОЗРАСТНЫМ ГРУППАМ")

print(age_analysis.to_string())


# 4. ОПРЕДЕЛЕНИЕ ЛИДЕРА

winner = age_analysis.loc[age_analysis['total_amount'].idxmax()]

print(f" САМАЯ ПЛАТЕЖЕСПОСОБНАЯ ГРУППА: {winner.name}")

print(f"   • Клиентов: {winner['unique_customers']:,}")
print(f"   • Транзакций: {winner['txn_count']:,}")
print(f"   • Общий объём: ₹{winner['total_amount']/1e6:,.2f}M")
print(f"   • Средний чек: ₹{winner['avg_txn_amount']:,.2f}")
print(f"   • Средний баланс: ₹{winner['avg_balance']:,.2f}")
print(f"   • На клиента: ₹{winner['amount_per_customer']:,.2f}")
print(f"   • Доля оборота: {winner['total_amount']/age_analysis['total_amount'].sum()*100:.1f}%")
```

Определение самой платежеспособной группы

Молодёжь обеспечивает массовый объём транзакций, а старшие возрастные группы представляют собой премиальный сегмент с высокой платёжеспособность ю.



Распределение транзакций по локации

```python
df_work = df.copy()

# Очистка локаций
df_work['custLocation'] = df_work['custLocation'].astype(str).str.strip().str.title()
df_work = df_work[~df_work['custLocation'].isin(['Nan', 'Na', 'Unknown', ''])]


# 2. АГРЕГАЦИЯ ПО ЛОКАЦИЯМ

location_stats = df_work.groupby('custLocation').agg({
    'transactionId': 'count',
    'transactionAmountInr': ['sum', 'mean', 'median'],
    'customerId': 'nunique',
    'custAccountBalance': 'mean'
}).round(2)

location_stats.columns = ['txn_count', 'total_amount', 'avg_txn_amount',
                          'median_txn_amount', 'unique_customers', 'avg_balance']

# Дополнительные метрики
location_stats['total_amount_million'] = location_stats['total_amount'] / 1e6
location_stats['txn_per_customer'] = location_stats['txn_count'] / location_stats['unique_customers']


# 3. ТОП-10 ПО КОЛИЧЕСТВУ ТРАНЗАКЦИЙ

top10_by_count = location_stats.nlargest(10, 'txn_count')


print(" ТОП-10 ЛОКАЦИЙ ПО КОЛИЧЕСТВУ ТРАНЗАКЦИЙ")

print(f"{'№':<3} {'Локация':<20} {'Транзакций':>14} {'Сумма (M ₹)':>14} {'Клиентов':>12}")


for i, (loc, row) in enumerate(top10_by_count.iterrows(), 1):
    print(f"{i:<3} {loc:<20} {row['txn_count']:>14,} {row['total_amount_million']:>13.2f}M {row['unique_custome
```

[Без за

```python
# 6. КЛЮЧЕВЫЕ ИНСАЙТЫ

print(" Анализ локации по кол-ву транзакций к сумме транзакций")

# Лидер по количеству
count_leader = top10_by_count.iloc[0]
print(f" Лидер по количеству: {top10_by_count.index[0]} ({count_leader['txn_count']:,} транзакций)")

# Лидер по сумме
amount_leader = top10_by_amount.iloc[0]
print(f" Лидер по объёму: {top10_by_amount.index[0]} (₹{amount_leader['total_amount_million']:.2f}M)")

# Концентрация
total_top10_count = top10_by_count['txn_count'].sum()
total_all_count = location_stats['txn_count'].sum()
top10_count_share = total_top10_count / total_all_count * 100

total_top10_amount = top10_by_amount['total_amount'].sum()
total_all_amount = location_stats['total_amount'].sum()
top10_amount_share = total_top10_amount / total_all_amount * 100

print(f"\n Концентрация топ-10:")
print(f"   • По количеству: {top10_count_share:.1f}% всех транзакций")
print(f"   • По сумме: {top10_amount_share:.1f}% всего объёма")

# Различия в рейтингах
count_locs = set(top10_by_count.index)
amount_locs = set(top10_by_amount.index)
in_both = count_locs & amount_locs
only_in_count = count_locs - amount_locs
only_in_amount = amount_locs - count_locs

print(f"\n Различия в топ-10:")
print(f"   • В обоих рейтингах: {len(in_both)} локаций")
if only_in_count:
    print(f"   • Только по количеству: {', '.join(only_in_count)}")
if only_in_amount:
    print(f"   • Только по сумме: {', '.join(only_in_amount)}")

# Средний чек
highest_avg_check = location_stats.loc[location_stats['avg_txn_amount'].idxmax()]
print(f"\n Самый высокий средний чек: {highest_avg_check.name} (₹{highest_avg_check['avg_txn_amount']:,.2f})")
```

Распределение транзакций по локации

Пять мегаполисов (Mumbai, New Delhi, Bangalore, Gurgaon, Delhi) формируют ядро бизнеса, обеспечивая основную долю транзакционной активности.

Анализ распределения времени транзакции

```python
# Проверяем формат transactionTime
print(f"\n Формат transactionTime:")
print(f"   Тип: {df_work['transactionTime'].dtype}")
print(f"   Примеры: {df_work['transactionTime'].head(10).tolist()}")
print(f"   Min: {df_work['transactionTime'].min()}, Max: {df_work['transactionTime'].max()}")

sample_time = df_work['transactionTime'].iloc[0]

# ОПРЕДЕЛЯЕМ ФОРМАТ И КОНВЕРТИРУЕМ
if sample_time > 1000000:  # Скорее всего миллисекунды
    print(f"\n Формат: МИЛЛИСЕКУНДЫ от начала суток")
    print(f"   Пример: {sample_time} мс")

    # Конвертируем миллисекунды в минуты
    df_work['minutes_from_midnight'] = (df_work['transactionTime'] / 1000 / 60).astype(int)

    # Извлекаем часы и минуты
    df_work['hour'] = (df_work['minutes_from_midnight'] // 60).astype(int)
    df_work['minute'] = (df_work['minutes_from_midnight'] % 60).astype(int)

    # Показываем пример конвертации
    print(f"\n Примеры конвертации:")
    print(f"   {df_work['transactionTime'].iloc[0]} мс → "
          f"{df_work['minutes_from_midnight'].iloc[0]} мин → "
          f"{df_work['hour'].iloc[0]:02d}:{df_work['minute'].iloc[0]:02d}")

elif 0 <= sample_time <= 2359:  # Формат ННММ
    print(f"\n Формат: ННММ (часы и минуты)")

    df_work['hour'] = (df_work['transactionTime'] // 100).astype(int)
    df_work['minute'] = (df_work['transactionTime'] % 100).astype(int)
    df_work['minutes_from_midnight'] = df_work['hour'] * 60 + df_work['minute']

else:
    print(f"\n  Неизвестный формат. Пробуем использовать transactionDateTime...")
    df_work['transactionDateTime'] = pd.to_datetime(df_work['transactionDateTime'], errors='coerce')
    df_work['hour'] = df_work['transactionDateTime'].dt.hour
    df_work['minute'] = df_work['transactionDateTime'].dt.minute
    df_work['minutes_from_midnight'] = df_work['hour'] * 60 + df_work['minute']
```

```python
# Фильтруем корректное время (0-23 часа)
df_work = df_work[(df_work['hour'] >= 0) & (df_work['hour'] <= 23)].copy()

# Определяем время суток
def get_time_of_day(hour):
    if 0 <= hour < 6:
        return 'Night (00-06)'
    elif 6 <= hour < 12:
        return 'Morning (06-12)'
    elif 12 <= hour < 17:
        return 'Afternoon (12-17)'
    elif 17 <= hour < 21:
        return 'Evening (17-21)'
    else:
        return 'Night (21-24)'

df_work['time_of_day'] = df_work['hour'].apply(get_time_of_day)

# 2. СТАТИСТИКА


print(" СТАТИСТИКА ВРЕМЕНИ ТРАНЗАКЦИЙ")


print(f"\n Распределение по часам:")
hourly_dist = df_work.groupby('hour').agg({
    'transactionId': 'count',
    'transactionAmountInr': ['sum', 'mean']
}).round(2)
hourly_dist.columns = ['txn_count', 'total_amount', 'avg_amount']
print(hourly_dist.to_string())

print(f"\n По времени суток:")
time_of_day_stats = df_work.groupby('time_of_day').agg({
    'transactionId': 'count',
    'transactionAmountInr': ['sum', 'mean']
}).round(2)
time_of_day_stats.columns = ['txn_count', 'total_amount', 'avg_amount']
time_of_day_stats['share'] = time_of_day_stats['txn_count'] / len(df_work) * 100
print(time_of_day_stats.to_string())
```

Анализ распределения времени транзакции

Транзакционная активность показывает временной пик: 63.1% всех операций приходится на период с 12:00 до 21:00, при этом максимальная нагрузка наблюдается в вечерние часы (17-21), на которые приходится 32.4% транзакций.

СПАСИБО

ЗА ВНИМАНИЕ